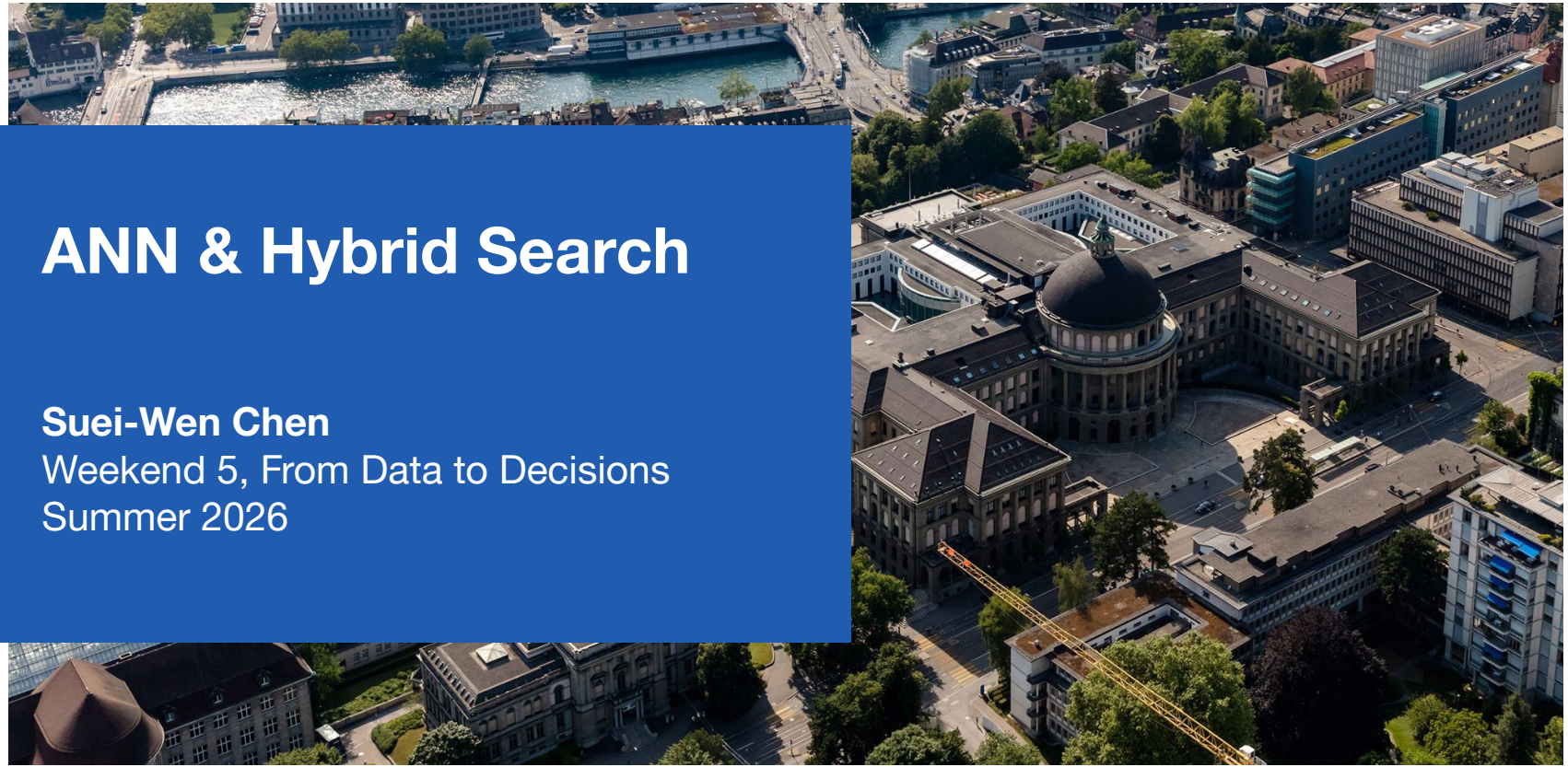
ETH zürich



ANN & Hybrid Search

Suei-Wen Chen

Weekend 5, From Data to Decisions
Summer 2026

Motivation

You run **ApexRetail**, an online retailer with an catalog of 10,000,000+ items

Promise: RAG-powered conversational AI assistant for

- **product recommendation** (shopping)
- **after-sales support** (answer technical questions)

Challenges

- **Imprecise recommendation:** low conversion rate
- **Incorrect support information:** high return rate
- **Latency:** User leaves the site if responses come out too slow

Example User Request

🗣️ `Find a compact espresso machine in stock under $200. The design should be modern and minimalistic.`

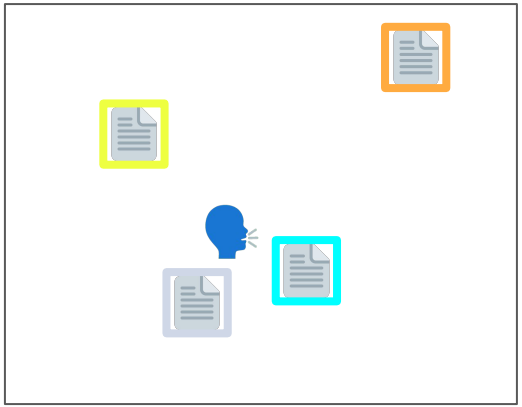
Database

Three factors at play: **Semantics, Keywords, Constraints**

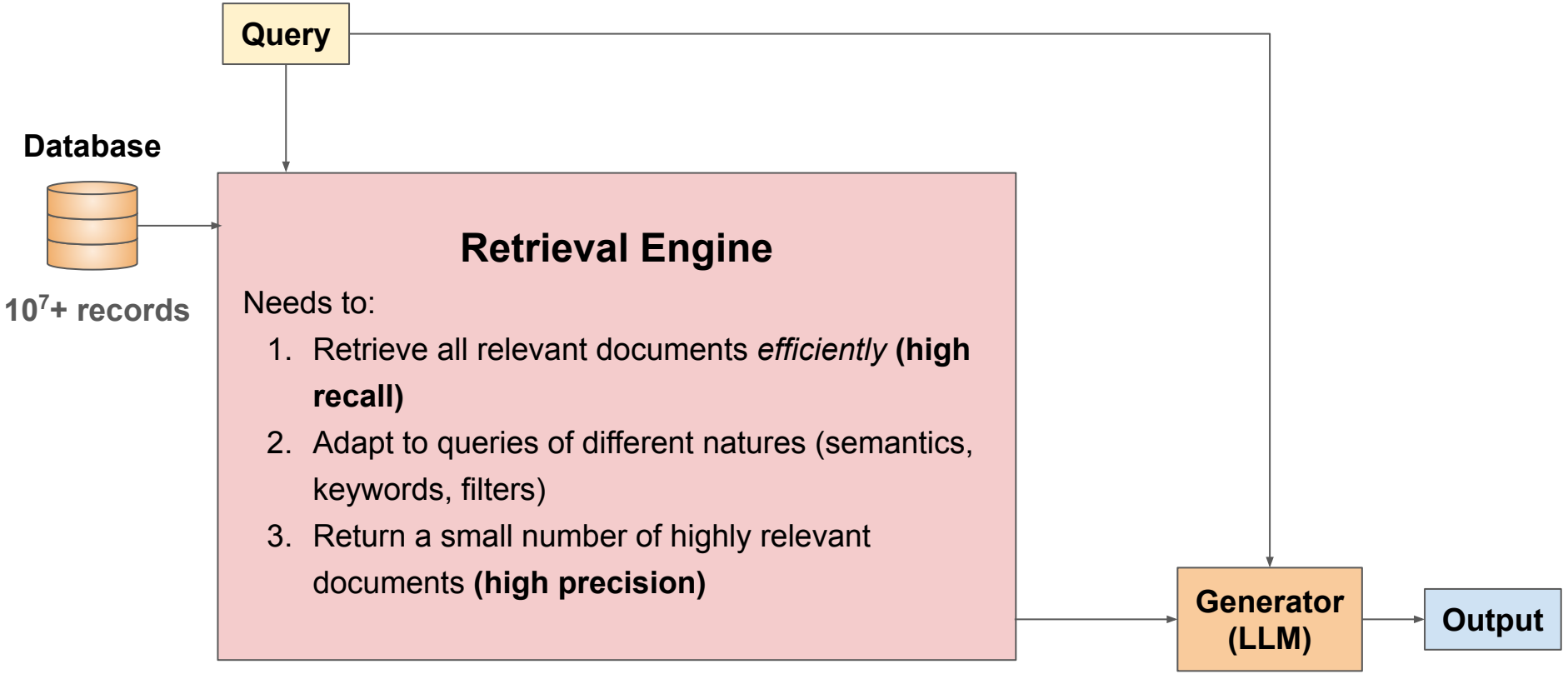
Tables (Relational)

ID	Product type	Stock	Price	Detailed Specs
A70D2V	Espresso Machine	8	$199	“... ornate and classical …”
31FFQ1	Espresso Machine	5	$199	“... ideal for small apartment …”
9DH2J2	Pod Machine	7	$180	“... ultra-small modern …”
00KLP3	Espresso Machine	0	$250	“... compact and modern …”

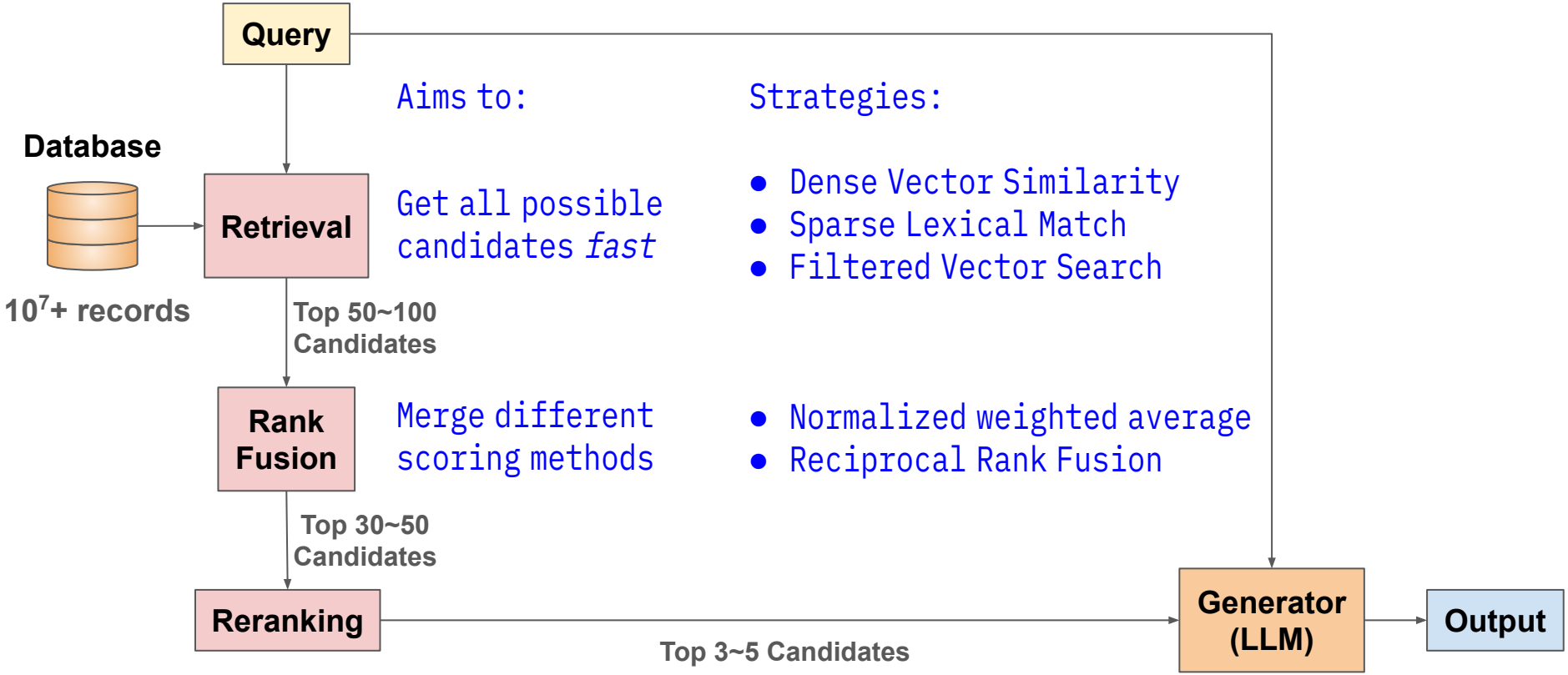
Vector DB (Semantic)



Roadmap: RAG with a Hybrid Search Pipeline



Roadmap: RAG with a Hybrid Search Pipeline



Roadmap: RAG with a Hybrid Search Pipeline

Nearest Neighbor (NN) Search is too expensive

🗣️ `Find the top 10 most trendy Christmas gift for 2026.`

Vector DB
(Product Specs)

Your vector DB contains N>10^7 records. ector dimension:

d=384 (all-MiniLM-L6-v2)
d=768 (BERT-base)
d=1536 (OpenAI text-embedding-3-small)
d=3072 (OpenAI text-embedding-3-large)

Computing all cosine similarities to get exact k-NNs takes **seconds**

Amazon found every 100 milliseconds in latency cost 1% in sales

Trade-off: Sacrifice some recall and compute ***approximate*** k-NNs for massive speedup

Approximate Nearest Neighbor (ANN) Search

Original Complexity is O(dN). Two ways to speed things up:

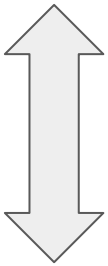
1) Compress the vectors (e.g. reduce dimension)
2) Reduce the number of distance computation

Methods:

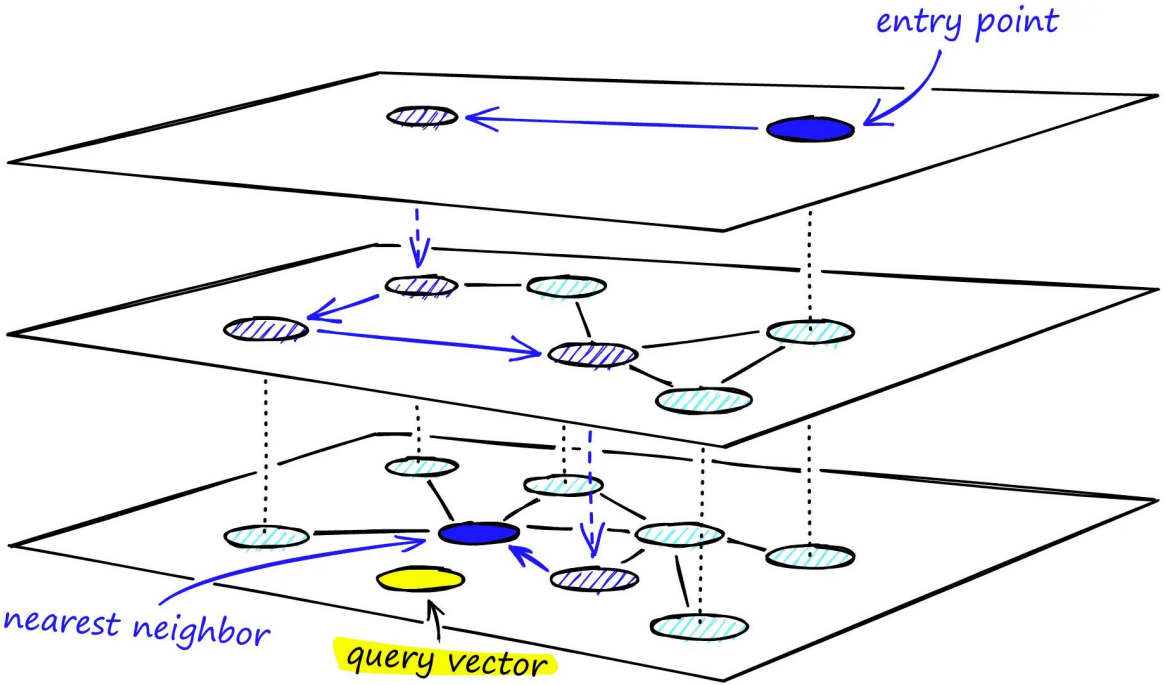
- **Vector Quantization (VQ):** Compression
- **Inverted File Index (IVF):** Cluster vectors into centroids
- **Product Quantization (PQ):** Clustering on sub-vectors
- **Hierarchical Navigable Small World (HNSW):** Graph index for fast traversal

HNSW: Much Faster Search O(d log N)

Top: sparse, long-range links (highways)



Bottom: dense, short-range links (local roads)



Neighbors are selected to avoid isolated clusters and ensure navigability

Roadmap: RAG with a Hybrid Search Pipeline

When Dense Retrieval Fails

🗣️ My coffee machine is showing the error code E034. How can I fix this?

Database

The sentence-transformer `all-MiniLM-L6-v2` gives `cos-sim(🗣️, 📄)=0.51` and `cos-sim(🗣️, 📄)=0.60`

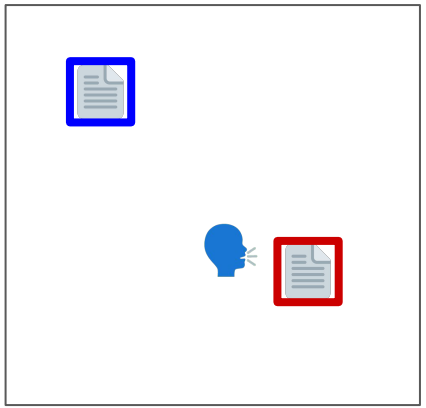
Relational DB (Structured)

ID	Product type	…	User Manual
31FFQ1	Espresso Machine		📄

Chunk 1: Code E034 means the group head is clogged. To fix this problem, remove the portafilter from the machine and clear any built-up residue.

Chunk 2: Code E035 means the machine is overheating. To fix this, turn off the main power switch and wait for five minutes to allow the boiler to cool completely.

Vector DB (Semantic)

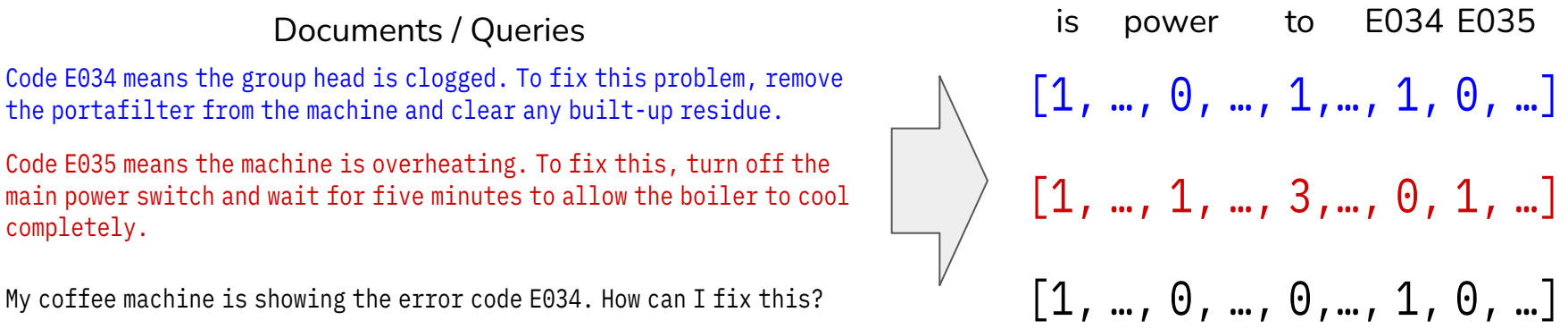


Fix: Sparse Lexical Match

Dense retrieval works well for conceptual queries but fails for ***exact entities***

- stock keeping unit (SKU) in retail
- error code in technical support
- specific clause numbers in legal documents

In sparse retrieval, documents/queries are represented as “bag of words”:



Count-based Scoring: TF-IDF (Term Frequency-Inverse Document Frequency)

From the bag-of-words of N documents we know:

$f_{t,d}$ = how many times term t appears in document d

$TF_{t,d} = \log(f_{t,d}+1)$

df_t = how many documents term t appears in

$IDF_t = \log(N/df_t)$

TF-IDF measures prominence of each term in a doc, relative to the entire corpus:

$TF\text{-}IDF_{t,d} = TF_{t,d} \times IDF_t$

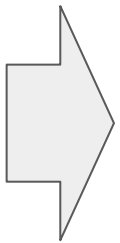
… then use cosine similarity

Documents / Queries

Code E034 means the group head is clogged. To fix this problem, remove the portafilter from the machine and clear any built-up residue.

Code E035 means the machine is overheating. To fix this, turn off the main power switch and wait for five minutes to allow the boiler to cool completely.

My coffee machine is showing the error code E034. How can I fix this?



is	power	to	E034	E035
[0.48, …, 0,	…, 0,	…,	1.44, 0,	…]
[0.48, …, 0.96,	…, 0,	…,	0, 1.44,	…]
[0.48, …, 0,	…, 0,	…,	1.44, 0,	…]

BM25: A better way to combine TF and IDF

$$\text{score}(q,d) = \sum_{t \in q} \overbrace{\frac{f_{t,d} \cdot (k+1)}{f_{t,d} + k\left(1 - b + \frac{b|d|}{\text{AvgDocLen}}\right)}}^{\text{Weighted TF}_{t,d}} \cdot \overbrace{\ln\left(\frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5} + 1\right)}^{\text{IDF}_t}$$

Two knobs:

- **k≥0: Term Frequency Saturation**, larger = slower saturation
- **0≤b≤1: Document Length Normalization**, larger = favoring shorter docs

Rule-of-thumb: k=1.2~2.0, b=0.75

Roadmap: RAG with a Hybrid Search Pipeline

HNSW/BM25 Alone Cannot Handle Structured Constraints

🗣️ `Find a compact espresso machine in stock under $200. The design should be modern and minimalistic.`

Database

Need to:
1. Perform semantic (vector) search
2. filter out records violating constraints

Tables (Relational)

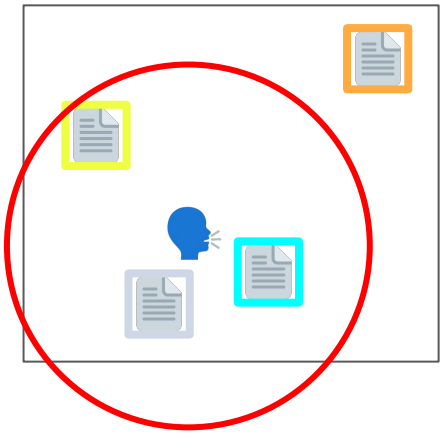
ID	Product type	Stock	Price	Detailed Specs
A70D2V	Espresso Machine	8	$199	"... ornate and classical …"
31FFQ1	Espresso Machine	5	$199	"... ideal for small apartment …"
9DH2J2	Pod Machine	7	$180	"... ultra-small modern …"
00KLP3	Espresso Machine	0	$250	"... compact and modern …"

Vector DB (Semantic)

Method #1: Post-filter (First ANNS, then filter)

🗣️ `Find a compact espresso machine in stock under $200. The design should be modern and minimalistic.`

Vector DB (HNSW)



search

k-ANNs

ID	Product type	Stock	Price
31FFQ1	Espresso Machine	5	$199
9DH2J2	Pod Machine	7	$180
00KLP3	Espresso Machine	0	$250

filter

Output

ID	…
31FFQ1	…

Issue: No (or not enough) ANNs pass through the filter for selective constraints

Method #2: Pre-filter (First Filter, then brute-force NNS)

🗣️ `Find a compact espresso machine in stock under $200. The design should be modern and minimalistic.`

Table (All 10^7+ records)

ID	…	Detailed Specs
…	…	
…	…	
…	…	
…	…	

filter

Qualifying Candidates

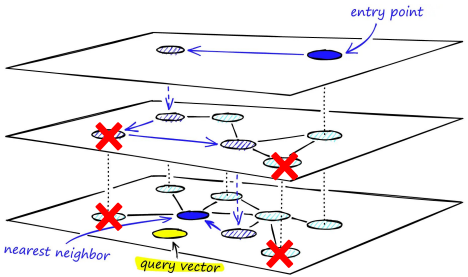
ID	Product type	Stock	Price
31FFQ1	Espresso Machine	5	$199
9DH2J2	Pod Machine	7	$180
00KLP3	Espresso Machine	0	$250

exact NN search

Output

ID	…
31FFQ1	…

Issue: Cannot use HNSW because dropping records disconnects the graph

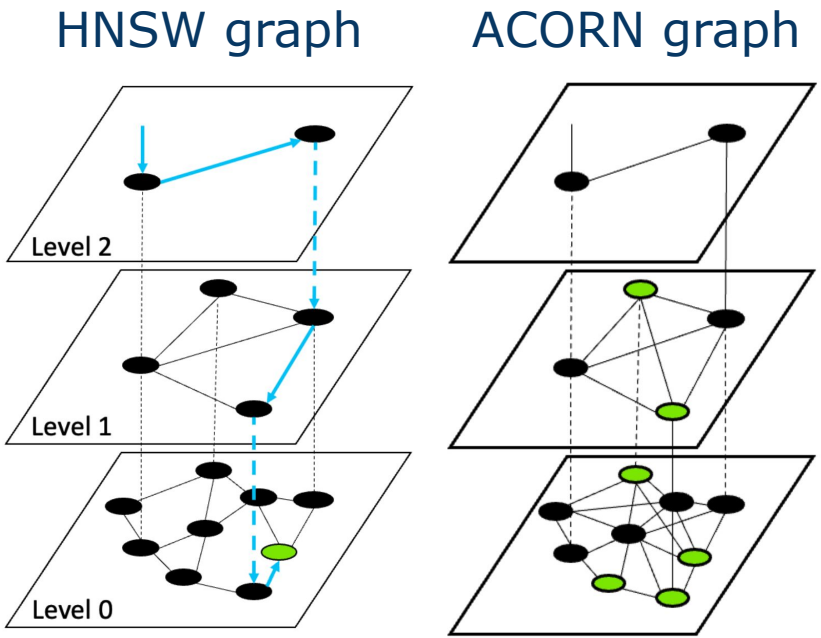


Method #3: Inline-filter (Filter-Compatible HNSW)

🗣️ `Find a compact espresso machine in stock under $200. The design should be modern and minimalistic.`

ACORN (ANN Constraint-Optimized Retrieval Network)

- Construct a denser HNSW graph (in a smart way) which remains connected upon filtering



Roadmap: RAG with a Hybrid Search Pipeline

Hybrid Search: Combine Dense & Sparse Retrieval

Same query, different search methods → incomparable scores

🗣️ `Find the best Black Friday smart home starter pack for elderly parents`

ID	cos-sim
L20Q13	0.92
38HJ2K	0.85
BHDD15	0.74
…	…
GK0023	0.49
…	…

ID	BM25
BHDD15	28.1
GK0023	22.5
38HJ2K	17.7
…	…
L20Q13	13.2
…	…

Idea 1: Normalize each score and take a weighted average

Idea 2: Ignore numeric scores and focus on the rankings → **Reciprocal Rank Fusion**

Reciprocal Rank Fusion (RFF)

$$\mathrm{RFF}(d) = \sum_{\text{metric } m} \frac{1}{r_m(d) + \text{const.}}$$

- If `d` is not in list `m`, $r_m(d) = \infty$
- Industry standard: $\text{const.} = 60$

Fusing different scoring methods gives us higher-quality candidates!

→ The top ones will be further fed into the reranker

Metric #1

ID	cos-sim	Rank $r_1(d)$
L20Q13	0.92	1
38HJ2K	0.85	2
BHDD15	0.74	3
…	…	…
GK0023	0.49	8
…	…	…

Metric #2

ID	BM25	Rank $r_2(d)$
BHDD15	28.1	1
GK0023	22.5	2
38HJ2K	17.7	3
…	…	…
L20Q13	13.2	13
…	…	…

Resources & Further Readings

- Hybrid search using vectors and full text in Azure AI Search
- Hybrid Search with Amazon OpenSearch Service
- Azure AI Search: Why Hybrid Search Beats Pure Vector
- Practical BM25 - Part 3: Considerations for Picking b and k1 in Elasticsearch
- ETH Vector Search in Databases for AI Seminar (HS25)